

Redux: State Management

What is Redux?

Redux is a **predictable state container** for JavaScript apps (often used with React). It centralizes application state, making it easier to manage and debug.

Core Concepts:

- **Store:** Single source of truth holding the entire app state.
- **Actions:** Plain JS objects describing what happened (e.g., `{ type: 'ADD_TASK', payload: 'Buy milk' }`).
- **Reducers:** Pure functions that take the current state + action, and return a new state.

```
const taskReducer = (state = [], action) => {  
  switch (action.type) {  
    case 'ADD_TASK': return [...state, action.payload];  
    default: return state;  
  }  
};
```

- **Dispatch:** Trigger state changes by sending actions (`store.dispatch(action)`).

Why Use Redux?

- Avoids **prop drilling** (passing state through multiple components).
- Enables **time-travel debugging** (Redux DevTools).
- Handles **async logic** via middleware (e.g., `redux-thunk`).

Redux vs. Context API

Feature	Redux	Context API
Use Case	Complex state	Simple, low-frequency updates
Performance	Optimized for frequent updates	Re-renders all consumers
Middleware	Supports <code>redux-thunk</code> , <code>saga</code>	None

2. State vs. Props in React

State

- **Definition:** Data managed **within a component** (e.g., form inputs, UI toggle state).
- **Mutable:** Updated via `setState` (class) or `useState` (hooks).

```
const [tasks, setTasks] = useState([]);
```

- **Scope:** Local to the component (unless lifted to a parent or Redux).

Props

- **Definition:** Data passed **from parent to child** (read-only).
- **Immutable:** Child components cannot modify props.

```
function TaskList({ tasks }) { // `tasks` is a prop
  return tasks.map(task => <div>{task}</div>);
}
```

- **Purpose:** Enables reusability and unidirectional data flow.

Key Difference

- **State:** Internal, mutable.
 - **Props:** External, immutable.
-

3. REST API

What is REST?

REST (Representational State Transfer) is an architectural style for designing networked applications using HTTP.

Core Principles:

- **Stateless:** Each request contains all needed info (no server-side sessions).
- **Resource-Based:** Data is exposed as **resources** (e.g., `/tasks`, `/users`).

- **HTTP Methods:**

- **GET** (Read)
- **POST** (Create)
- **PUT/PATCH** (Update)
- **DELETE** (Delete)

Example: To-Do App API

Endpoint	Method	Description
/api/tasks	GET	Fetch all tasks
/api/tasks	POST	Add a new task
/api/tasks/:id	DELETE	Remove a task

REST vs. GraphQL

Feature	REST	GraphQL
Data Fetch	Multiple endpoints	Single endpoint, flexible queries
Overfetching	Possible (fixed responses)	Avoids (client specifies fields)

4. WebSockets (Real-Time Communication)

What are WebSockets?

A protocol providing **full-duplex** (bidirectional) communication over a single TCP connection, ideal for real-time apps (e.g., chat, live updates).

How It Works:

1. **Handshake:** Client connects via HTTP → Upgrades to WebSocket (**ws://** or **wss://**).
2. **Persistent Connection:** Server/client can send messages anytime without HTTP overhead.

Implementation (Socket.io)

Server:

```
const io = require('socket.io')(server);
io.on('connection', (socket) => {
  socket.on('newMessage', (msg) => io.emit('message', msg));
});
```

Client:

```
socket.emit('newMessage', 'Hello!');
socket.on('message', (msg) => console.log(msg));
```

Use Cases

- Chat apps
- Live notifications
- Collaborative editing (e.g., Google Docs)

5. JWT Authentication

What is JWT?

JSON Web Tokens (JWT) are **stateless** tokens for securely transmitting user identity between client/server.

Structure (3 Parts):

1. **Header:** Algorithm + token type ({ "alg": "HS256", "typ": "JWT" }).
2. **Payload:** Claims (e.g., { "userId": "123", "exp": 1735689600 }).
3. **Signature:** Verifies token integrity (HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret)).

Workflow:

1. **Login:** User submits credentials → Server validates → Returns JWT.
2. **Storage:** Client stores JWT in `localStorage` or cookies.
3. **Requests:** JWT sent in `Authorization: Bearer <token>` header.
4. **Verification:** Server checks signature + expiry before granting access.

Security Notes

- **Never store sensitive data** in JWT (payload is base64-encoded, not encrypted).
- Use **HTTPS** to prevent token theft.
- Set short expiry times + refresh tokens.

Summary Table

Concept	Purpose	Key Features
Redux	Global state management	Single store, actions, reducers
State/Props	Component data handling	State: mutable; Props: immutable
REST API	Client-server communication	Stateless, resource-based, HTTP methods
WebSockets	Real-time bidirectional comms	Persistent connection, low latency
JWT	Secure authentication	Stateless, self-contained, scalable

Why These Matter in MERN?

- **Redux**: Manages complex state (e.g., user auth, tasks).
- **REST API**: Connects React frontend to Node.js backend.
- **WebSockets**: Enables real-time features (chat, notifications).
- **JWT**: Secures API endpoints without server-side sessions.

These technologies form the backbone of modern full-stack apps, ensuring **scalability**, **real-time capabilities**, and **security**.

Redux Explained Simply (Like You're 10)

Imagine Redux as a **magical notebook** that keeps track of **everything happening in your app** (like a to-do list, user login status, or theme color). Instead of scattering this info across different components, Redux stores it all in **one central place**—the "**Store**."

How Redux Works (Step-by-Step)

1. The Store (The Big Notebook)

- This is a **single JavaScript object** that holds the **entire state** of your app.
- Example:

```
{
  tasks: ["Buy milk", "Walk dog"],
  isLoggedIn: true,
  theme: "dark"
}
```

2. Actions (Messages Asking for Changes)

- When something happens (e.g., user adds a task), you send an **action**—a simple note describing what happened.
- Example action:

```
{
  type: "ADD_TASK",    // What happened?
  payload: "Call mom"  // Extra info (the new task)
}
```

3. Reducers (The Rulebook)

- A **reducer** is a function that takes the **current state** + an **action**, and returns the **new state**.
- Example reducer:

```
const taskReducer = (state = [], action) => {
  switch (action.type) {
    case "ADD_TASK":
      return [...state, action.payload]; // Add new task
    case "DELETE_TASK":
      return state.filter(task => task !== action.payload);
    default:
      return state; // If no action matches, do nothing
  }
}
```

```
}  
};
```

4. Dispatch (Sending the Action)

- To update the state, you **"dispatch"** an action:

```
store.dispatch({ type: "ADD_TASK", payload: "Call mom" });
```

- Redux **automatically** calls the reducer, updates the store, and notifies React to re-render.

Real-Life Example (To-Do App)

1. Initial State:

```
{ tasks: [] } // No tasks yet
```

2. User adds "Buy milk":

- Dispatch:

```
dispatch({ type: "ADD_TASK", payload: "Buy milk" });
```

- Reducer updates state:

```
{ tasks: ["Buy milk"] }
```

3. UI Updates: React sees the change and **re-renders** the to-do list.

Why Use Redux?

- **No Prop Drilling:** Avoid passing state through 10 components just to use it in the 11th.
- **Predictable Debugging:** Every state change is logged (Redux DevTools lets you "time travel").
- **Scalable:** Great for large apps (e.g., e-commerce carts, user dashboards).

Redux vs. React State

Feature	Redux	React State (<code>useState</code>)
Scope	Global (entire app)	Local (single component)
Updates	Via <code>dispatch(action)</code>	Via <code>setState</code> or <code>useState</code>
Best For	Complex apps	Simple component state

When Should You Use Redux?

- ✓ **Large apps** with shared state (e.g., user auth, shopping cart).
- ✓ **Frequent state updates** (e.g., real-time chat).
- ✗ **Small apps** where React state is enough.

Summary

- **Store** = Big notebook holding state.
- **Actions** = Notes requesting changes (e.g., "Add task").
- **Reducers** = Rules that update the notebook.
- **Dispatch** = Sending the note to Redux.

Redux keeps your app **organized** and **predictable**—like a well-managed library instead of a messy desk! 📚✨